Deduplicação de leituras de sensores

Integrantes: Henry Rodrigues, Rafael Maluf e Matheus.

Contexto, contrato e restrições

O domínio tem D=65536 valores (0 a 65535). n é a quantidade de leituras e u a quantidade de distintos: $0 \le u \le \min(n,D)$. Se n>D, necessariamente existem repetições, mas não necessariamente todos os D valores. A entrada do Heap Sort C é um ponteiro uint16_t* e um tamanho size_t; o bitmap Python recebe array('H'), de 2 bytes por elemento. Nos experimentos, o buffer do array é compartilhado com C sem cópia por ctypes. As funções modificam a entrada e retornam u; apenas a[0:u] é a saída válida. Heap devolve ordem crescente; bitmap mantém primeiras ocorrências. A entrada vazia retorna zero. São exatamente duas estratégias implementadas; conjuntos e ordenação nativa aparecem somente na infraestrutura de validação.

Embarca permite 256 bytes extras ao vetor já residente, incluindo a pilha das funções. Datacenter exige $n=10^7$ em menos de 2 s na máquina da disciplina. A solução Embarca foi implementada em C para atender ao orçamento físico; o bitmap e a automação permanecem em Python. O build avaliado usa frames de 48 B para heap_unique e 32 B para sift, mais uma margem conservadora de 16 B: 96 B extras. Não há recursão nem alocação dinâmica no algoritmo. O mesmo objeto C é usado no teste nativo sem interpretador e na biblioteca da infraestrutura Python. O limite independe de n nesse build; requer nova avaliação em outro compilador/ABI. A seção 6 permite escolha da linguagem, mas os objetivos e a rubrica citam Pascal: permanece necessária confirmação do professor e aprovação do par.

1. Força bruta

Buscar cada leitura no prefixo de únicos e anexar se ausente resolve o problema exatamente. O máximo de comparações para n e u fixos é $u(u-1)/2+(n-u)u$: as primeiras inserções custam 0 até $u-1$ comparações; as repetições podem custar u. Melhor $\Theta(n)$ com iguais; pior genérico $\Theta(n^2)$ se $D \ge n$; limite $O(nu)$ e, para D fixo, $O(nD)$. Não existe média sem distribuição; $nu/2$ é apenas aproximação de posições uniformes após formar o conjunto. Pilha e escalares O(1) palavras, heap/disco zero no modelo de implementação de baixo nível. Objetos Python não recebem essa garantia em bytes. Embarca pode usar uma versão de baixo nível auditada; Datacenter evita até cerca de $3{,}28\times10^{11}$ comparações para $n=10^7$ e $u \approx D$ (328 s a 1 ns/comparação hipotético).

2. Ordenação e varredura — Heap Sort implementado

Construir max-heap custa $\Theta(n)$, pois a soma das alturas dos nós é $O(n)$. Cada extração coloca o máximo no sufixo definitivo e faz uma descida $O(\log n)$; total $O(n \log n)$, seguido de compactação $\Theta(n)$. Com chaves distintas, permutações uniformes têm tempo médio $\Theta(n \log n)$, também atingido no pior caso genérico. Todos iguais encerram cada descida cedo: melhor $\Theta(n)$. Não se aplica o limite inferior de n chaves distintas quando n>D. A pilha de chamadas tem profundidade constante, sem recursão; a implementação não copia o vetor. Na implementação C, índices usam size_t e leituras usam uint16_t, com número fixo de temporários. A fonte, os frames static do compilador e o objeto sem chamadas externas fundamentam o limite local de 96 B. Não se confunde O(1) em palavras com uma constante física: ela foi conferida neste binário.

A descida troca com o maior filho, preservando as subárvores já em heap; a violação possível desce e termina. A extração mantém prefixo em heap e sufixo ordenado. Depois da ordenação, valores diferentes do último emitido são exatamente os únicos. O índice de escrita nunca avança além da leitura. Quick Sort recursivo usual tem média $\Theta(n \log n)$, pior $\Theta(n^2)$ e pilha dependente de n; Merge Sort usual necessita vetor auxiliar $\Theta(n)$. A escolha de Heap Sort C permite manter o vetor no próprio lugar e provar o limite de RAM no build avaliado. A ponte heap.py não implementa outra deduplicação; apenas chama a função C, e não é parte do firmware.

3. Hash/conjunto

Consultar e inserir só inéditos dá tempo esperado $\Theta(n)$, assumindo boa dispersão, carga controlada e custo $O(1)$ por hash. Redimensionamentos têm custo amortizado. Colisões adversas podem causar $O(nu)$, ou $\Theta(n^2)$ com n distintos colidentes. Heap $\Theta(u+1)$, pilha constante na variante iterativa, disco zero. Para D fixo, armazenam-se no máximo D chaves, não n entradas. O modelo nominal usa 16 B por distinto mais 128 B; não é o layout de set em CPython. Não garante os 256 B da Embarca; Datacenter poderia usar, mas bitmap evita colisões e tem domínio pequeno conhecido.

4. Árvore balanceada

Buscar/inserir em AVL ou Red-Black mantém altura $O(\log(u+1))$, dando limite $O(n \log(u+1)+u)$. Melhor $\Theta(n)$ com iguais; buscas uniformes entre u chaves têm média $\Theta(\log(u+1))$; sem distribuição não se define média única. Construção com $u \approx n$ no modelo usual de comparações tem custo $\Theta(n \log n)$. Nós $\Theta(u)$, administração constante no heap; pilha $O(\log(u+1))$ recursiva ou $O(1)$ com pais em versão iterativa; disco zero. Modelo nominal de 32 B por nó, não medição Python. Embarca falha para u grande; Datacenter pode preferir quando necessita ordenação dinâmica em domínio amplo, requisito ausente aqui.

5. Bitmap implementado

bytearray(8192) inicializa exatamente 65536 bits. Para x, o byte está em x>>3 (0 a 8191) e a máscara é 1<<(x&7) (deslocamento 0 a 7). Antes de cada iteração, os bits representam os valores já lidos e o prefixo de saída contém suas primeiras ocorrências. Emitir apenas se o bit estava zero mantém exatidão; escrever atrás da leitura não apaga o futuro. Tempo $\Theta(n+\text{ceil}(D/8))$ no modelo de bytes, nos três casos; D fixo dá $\Theta(n+1)$. Além dos 8192 B de payload no heap há cabeçalho do bytearray, índices e execução Python; pilha tem profundidade constante e disco zero. Viola o orçamento Embarca só com os bits. É a recomendação Datacenter, condicionada ao teste real na referência e aos novos resultados Python.

6. Bloom

Usar m bits e k hashes, emitir quando algum bit está zero e então marcar, pode perder um valor inédito por falso positivo. Não é uma duplicata restante. Sob hashes uniformes independentes e aproximação de independência dos bits, $p \approx (1-\exp(-ku/m))^k$. O ótimo contínuo é $k=(m/u)\ln 2$, $m=-u \ln p/(\ln 2)^2$. Para capacidade D e p-alvo=1%, k=7 exige $m \geq -7D/\ln(1-0{,}01^{(1/7)})$; arredondando a bytes, 78586 B dão $p \approx 0{,}0099996885$ no modelo. Arredondar k sem ajustar m pode exceder 1%. Quando u=D não há consulta inédita no domínio: dimensionar para D é conservador para os estados anteriores. A taxa por consulta não é a probabilidade de saída inteira correta, nem garantia determinística. Uma implementação escolhida precisaria medir a taxa; aqui Bloom não foi implementado.

A variante que avalia sempre k hashes custa $\Theta(nk+\text{ceil}(m/w))$; para k fixo é linear. Heap $\Theta(m)$ bits mais escalares, pilha constante, disco zero. O filtro do modelo é fixo em 78586 B para tempo e memória. Embarca não comporta; Datacenter teria aproximação em uma estrutura maior que o bitmap exato, portanto não é a escolha.

7. Bogosort

Checar ordem e embaralhar uniformemente até ordenar, depois compactar, é exato quando termina. Com multiplicidades c_j, $q=\prod c_j!/n!$ é a probabilidade de uma permutação rotulada estar ordenada. Para entrada desordenada e tentativas independentes, $E[\text{tentativas}]=1/q$ e tempo esperado $\Theta(n/q)$. Entrada já ordenada custa $\Theta(n)$; para $q<1$ não há teto finito do número de tentativas. Com iguais $q=1$ e basta checar. Pilha, heap de trabalho e RNG podem ter estado constante em implementação iterativa; disco zero, sem garantia de bytes em Python.

n=1000 distinto é válido: $\log_{10}(1000!) \approx 2567{,}604644222$. A 1 ns/elemento/tentativa hipotético, o tempo seria cerca de $4{,}02 \times 10^{2561}$ s. Para n>D usamos o multinomial com multiplicidades balanceadas, não n! sem

correção. Frequências balanceadas maximizam permutações: transferir uma unidade de a para b com $a \geq b+2$ reduz o denominador pelo fator $(b+1)/a$. É impraticável em geral; já ordenado e n minúsculo são exceções. Embarca não tem teto temporal formal no PDF; Datacenter não recebe garantia de 2 s.

8. Subconjuntos

Enumerar incluir/excluir cada posição visita 2^n folhas e $2^{n+1}-1$ nós. Recuperar os singletons e emitir só valores ainda ausentes fornece uma solução deliberadamente redundante. Posições são distintas mesmo quando valores coincidem. Só visitar custa $\Theta(2^n)$ tempo; recursão tem profundidade n+1 e pilha $\Theta(n)$, e pode ter heap zero em streaming. Materializar todos os elementos custa $\Theta(n2^n)$, pois $\Sigma t \cdot \text{binom}(n,t) = n2^{n-1}$. Com inteiros de 2 B, o payload mínimo é $n2^n$ B, além dos descritores. As curvas adotam materialização para tempo e RAM, com layouts nominais explícitos.

Para n=1000: $2^{1000} \approx 1{,}0715 \times 10^{301}$ subconjuntos, payload $\approx 1{,}0715 \times 10^{304}$ B, e pilha de 16016 B sob hipótese otimista de 16 B/frame. Só enumerar a 10^9 folhas/s exigiria $\approx 1{,}0715 \times 10^{292}$ s. Viola memória Embarca e viabilidade temporal geral; em Python o limite de recursão também impede grandes profundidades. Para $n \leq 10$ pode servir em outro problema combinatório.

9. Unário em disco

Definimos cada registro x como x bytes '1' e um byte '0', necessário inclusive para zero. Gravar tudo não deduplica; a variante exata compara o comprimento de cada registro com todos os anteriores, sem parada antecipada, e compacta a saída no vetor. $S = \Sigma(x_j+1)$, com $n \leq S \leq 65536n$. Só serializar custa $\Theta(S)$, linear em n para domínio fixo. Na deduplicação, o tráfego é $V = \Sigma(n-j+1)(x_j+1)$; o registro j é gravado uma vez e relido $n-j$ vezes. Se $E[x]=32767{,}5$, $E[V]=32768{,}5n(n+1)/2$. Essa variante tem tráfego $\Theta(n^2)$ para D fixo. Contadores e buffer podem usar RAM constante em modelo de baixo nível, mas bibliotecas/interpretador não estão garantidos; disco $\Theta(S)$, pilha constante.

Para $n=10^7$: S esperado=327685000000 B (327,685 GB decimais), pior=655360000000 B. Só escrever a 1 GB/s hipotético custa 327,685 s. O tráfego esperado completo é 1638425163842500000 B, cerca de $1{,}638 \times 10^9$ s à mesma taxa, sem seeks/CPU. É exponencial nos bits b de um valor que pode chegar a 2^b-1, não em n com b=16 fixo. Não há teto formal de disco no PDF: o volume é uma limitação prática. n=1, x=0 exige só um byte.

10. Matriz n×n

Registrar $M_{ij} = (a_i = a_j)$ para todos os pares e emitir i quando não há igual $j<i$ resolve exatamente. A construção custa $\Theta(n^2)$ no melhor, médio e pior caso porque todas as células são exigidas pela auditoria. Heap n^2 B em bytes ou $\text{ceil}(n^2/8)$ B em bits, além de escalares; pilha constante e disco zero na variante residente. Para $n=10^7$: 10^{14} células, 100 TB ou 12,5 TB empacotados (11,37 TiB). A 1 ns/célula hipotético, 100000 s (27,78 h). n=1000 já exige 125000 B em bits, acima de 256 B; n=10 exige 13 B de payload e pode ser utilizável para auditoria pequena. Datacenter não tem limite formal de memória, mas o custo operacional e o tempo do modelo são impeditivos.

Modelos e leitura dos gráficos

As oito alternativas NÃO foram implementadas ou medidas. Modelamos $c=10^{-9}$ s por operação nominal, não calibrada: força bruta $cn \cdot \max(1, E[u]/2)$; hash 5cn; árvore $5cn \cdot \log_2(E[u]+1)$; Bloom 7cn+78586c; Bogosort $cn \cdot n!/\prod c_j!$; subconjuntos $cn2^n$; unário $E[V]/10^9$ B/s; matriz cn^2. $E[u] = D[1-(1-1/D)^n]$ sob amostragem uniforme independente, hipótese aproximada pelo gerador. Substituir E[u] numa função não linear não produz automaticamente E[T]; nu/2 também não representa exatamente a fase inicial de formação dos únicos.

Os custos nominais de RAM em bytes são: força bruta/Bogosort/unário 128; hash 16E[u]+128; árvore 32E[u]+128; Bloom 78586+128; subconjuntos $(n+16)2^n+16(n+1)+128$; matriz $\text{ceil}(n^2/8)+128$. São modelos otimistas de representação compacta, NÃO layouts Python. Para estruturas astronômicas,

endereços de 64 bits já não bastariam; só o payload mínimo prova inviabilidade. As curvas são cálculos hipotéticos, não previsões calibradas de C ou Python. As medições reais se identificam por pontos e linhas contínuas; alternativas usam tracejado. Caches, SO, distribuição e interpretador afetam os tempos e não foram isolados por experimento causal.

Cada figura está em um arquivo próprio: tempo_medido.png (somente as duas medidas), tempo_comparativo.png (faixa prática com oito modelos), tempo_completo.png (todos os expoentes), memoria_algoritmos.png (limite GCC para C e picos Python nas três escalas), memoria_comparativa.png (modelos compactos e evidências das duas implementações) e armazenamento.png. Nos painéis de expoentes, o eixo vertical é E em quantidade=10^E; uma escala simétrica comprime expoentes gigantes sem eliminá-los. Para C, a figura usa o limite estático GCC com margem, válido para todo n nesse objeto e acompanhado de execuções corretas nos três tamanhos. Para o bitmap Python, usa picos de tracemalloc em execuções completas, separadas dos tempos; não é toda a RAM interna ou pilha nativa. Os métodos distintos estão na legenda e não se somam ao RSS.

A comparação de tempo inclui implementações em linguagens distintas. Nos vetores menores, o custo do interpretador e da ponte ctypes pode pesar; para n grande, a complexidade e os acessos ao vetor também importam. A razão observada não isola o efeito exclusivo dos algoritmos. A recomendação Embarca se baseia no limite de memória, não em cumprir o prazo que pertence à Datacenter.

Referências

- Cormen, Leiserson, Rivest e Stein. Introduction to Algorithms. 4ª ed., MIT Press, 2022. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

- Bloom, B. H. Space/Time Trade-offs in Hash Coding with Allowable Errors. CACM 13(7), 1970, 422–426. https://doi.org/10.1145/362686.362692

- Broder, A.; Mitzenmacher, M. Network Applications of Bloom Filters: A Survey. Internet Mathematics 1(4), 2004, 485–509. https://doi.org/10.1080/15427951.2004.10129096

- GCC. Developer Options: -fstack-usage. https://gcc.gnu.org/onlinedocs/gcc/Developer-Options.html

- Python Software Foundation. array, tracemalloc e time. https://docs.python.org/3/library/array.html ; https://docs.python.org/3/library/tracemalloc.html ; https://docs.python.org/3/library/time.html

Metodologia e resultados — C e Python

Um aquecimento e três rodadas por algoritmo/tamanho. Heap Sort é C nativo; bitmap é Python. Ambos usam as mesmas leituras de random.Random(20261108).getrandbits(16), reiniciado por escala. A entrada de teste é array("H") com elementos nativos de 2 B. O adaptador ctypes compartilha seu buffer com C, sem copiar. A preparação desse ponteiro, geração, cópia e validação ficam fora do cronômetro. O tempo C inclui a chamada via ctypes e seu pequeno custo de conversão/retorno; não se subtrai uma estimativa desse custo, que é relevante nos vetores menores. Inicialização do bitmap e compactação ficam dentro do tempo.

Ambiente: 3.12.14 (main, Aug 25 2026, 14:00:49) [Clang 22.1.3]. Linux-6.18.57-x86_64-with-glibc2.41. model name : AMD EPYC 9V74 80-Core Processor. Compilador C: gcc (Debian 14.2.0-19) 14.2.0. Flags: -std=c11 -O3 -Wall -Wextra -Wpedantic -Werror -fno-inline -fno-omit-frame-pointer -mno-red-zone -fno-tree-loop-distribute-patterns -fno-ipa-cp -fstack-usage -fPIC.

Algoritmo	Linguagem	n	u	Mediana (s)	Mínimo (s)	Máximo (s)
heap	C	1000	996	0.000045	0.000038	0.000051
bitmap	Python	1000	996	0.000162	0.000160	0.000270
heap	C	100000	51281	0.008542	0.008052	0.010236
bitmap	Python	100000	51281	0.014614	0.014425	0.015011
heap	C	10000000	65536	2.092282	2.000067	2.513517
bitmap	Python	10000000	65536	1.017508	0.986792	1.092296

Validação independente ocorreu após todas as rodadas, fora da janela. Os testes C nativos passaram 3294 casos, além de NULL/0 e resposta conhecida, inclusive com AddressSanitizer/UndefinedBehaviorSanitizer em build separado. Incluem sentinelas, enumeração pequena, domínio completo e quatro distribuições em cada escala obrigatória. Os testes Python verificam a chamada C e o bitmap com set/sorted/dict como referências exclusivas de infraestrutura. Asserções ficam habilitadas. Sanitizadores não fornecem os tempos ou frames do build otimizado.

Memória extra: métodos distintos e recursos separados

Algoritmo	n	u	Extra pelo método (B)	Método	Entrada (B)
bitmap	1000	996	8553	Pico tracemalloc	2000
heap	1000	996	96	Limite GCC + margem	2000
bitmap	100000	51281	8553	Pico tracemalloc	200000
heap	100000	51281	96	Limite GCC + margem	200000
bitmap	10000000	65536	8553	Pico tracemalloc	20000000
heap	10000000	65536	96	Limite GCC + margem	20000000

Os três registros C correspondem a execuções completas validadas nos três tamanhos, com o mesmo limite estático do binário. Não são três picos de tracemalloc. O bitmap recebeu uma rodada completa rastreada por tamanho, separada dos tempos; entrada, relógio e validação ficaram fora do rastreamento. Seus picos não incluem toda a memória interna do interpretador e não provam todos os casos possíveis. RSS é o máximo do processo inteiro de testes; o payload calculado da entrada é 2n B, sem cabeçalho ou folga do array. Não se deduz memória extra subtraindo o vetor do RSS.

Comprovação do orçamento Embarca

A solução embarcada é src/heap.c, que pode ser ligada diretamente a um programa C, sem interpretador. Os executáveis de teste C usam o mesmo objeto do algoritmo usado na biblioteca. heap.py é apenas adaptador para a infraestrutura Python do computador; seus objetos, ctypes e validação não fazem parte do firmware proposto. Não se afirma que um processo Python inteiro caiba em 256 B.

O GCC gerou frames de tamanho static, conforme -fstack-usage:

```
src/heap.c:5:13:sift    32  static
src/heap.c:14:8:heap_unique 48  static
```

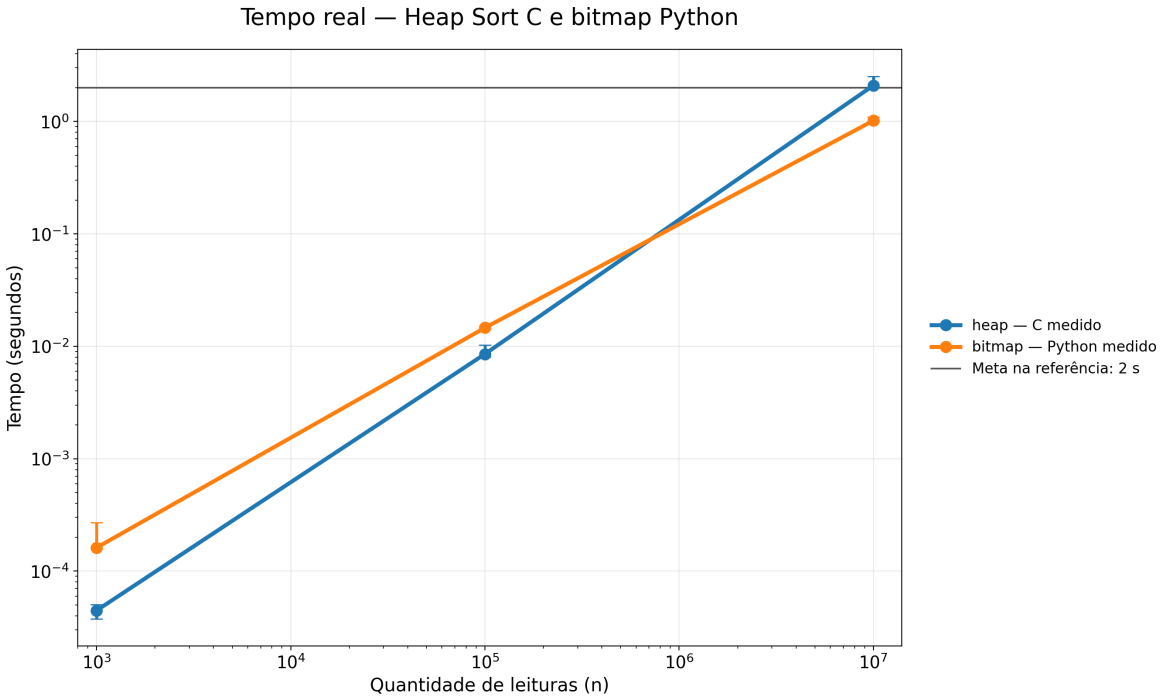
O caminho mais profundo é heap_unique → sift: 48+32=80 B. Acrescentando uma margem conservadora de 16 B na fronteira de chamada/ABI, obtemos 96 B, abaixo dos 256 B. Os frames do compilador incluem armazenamento de locais/temporários efetivamente colocado na pilha, alinhamento, salvamento de registradores e retorno conforme esse build; variáveis mantidas em registradores não alocam RAM adicional. Não se trata de uma soma incompleta de sizeof de variáveis.

Não há recursão, cópia do vetor ou alocação dinâmica no algoritmo. A fonte e a desmontagem foram conferidas; nm -u do objeto não apresenta símbolos externos. O heap dinâmico do algoritmo é zero. memoria.json preserva o relatório bruto do compilador, desmontagem, flags e hashes do objeto e biblioteca. O limite independe de n porque o caminho de chamadas é fixo e os frames são static. O build falha na coleta se a soma com margem passar de 256 B.

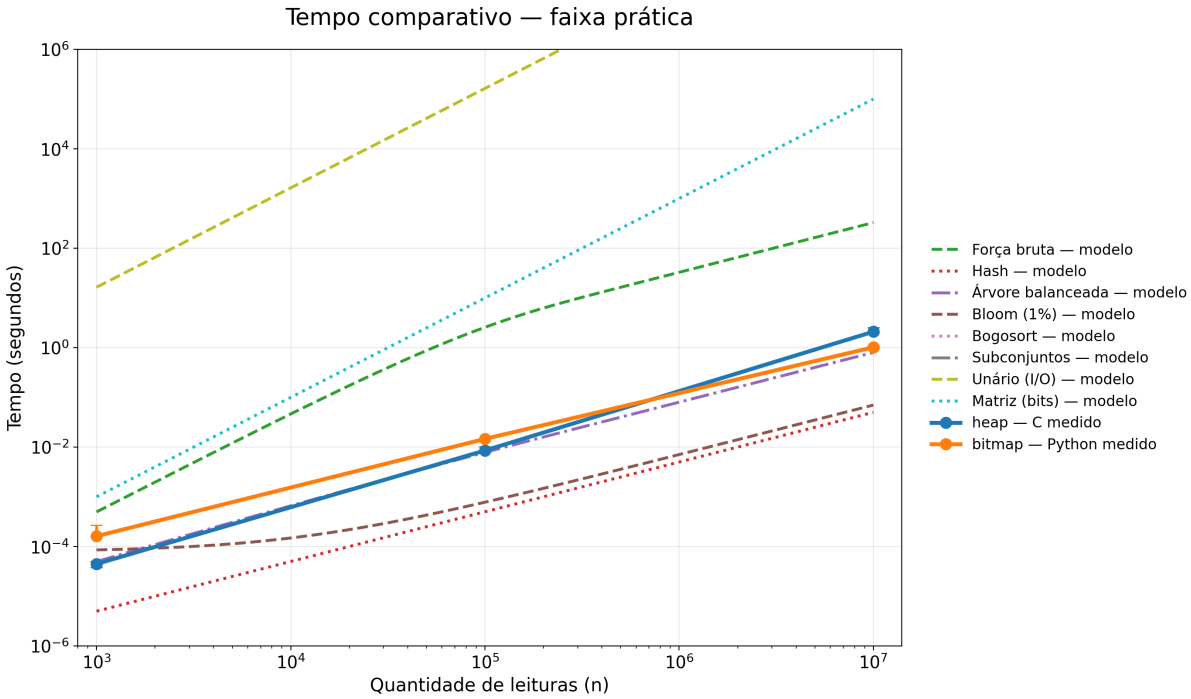
Essa comprovação vale para o GCC, ABI x86-64 e flags registrados, incluindo os três n testados. Outro alvo/compilador exige recompilar e auditar novamente; pilha do chamador e interrupções precisam de reserva própria no firmware real. O enunciado pressupõe o vetor de entrada já residente. Nenhum dispositivo embarcado específico foi informado, então não se declara validação de uma placa não testada.

O bitmap levou 1.017508 s de mediana local em $n=10^7$. A mediana ficou abaixo de 2 s localmente; isso não certifica a máquina da disciplina.

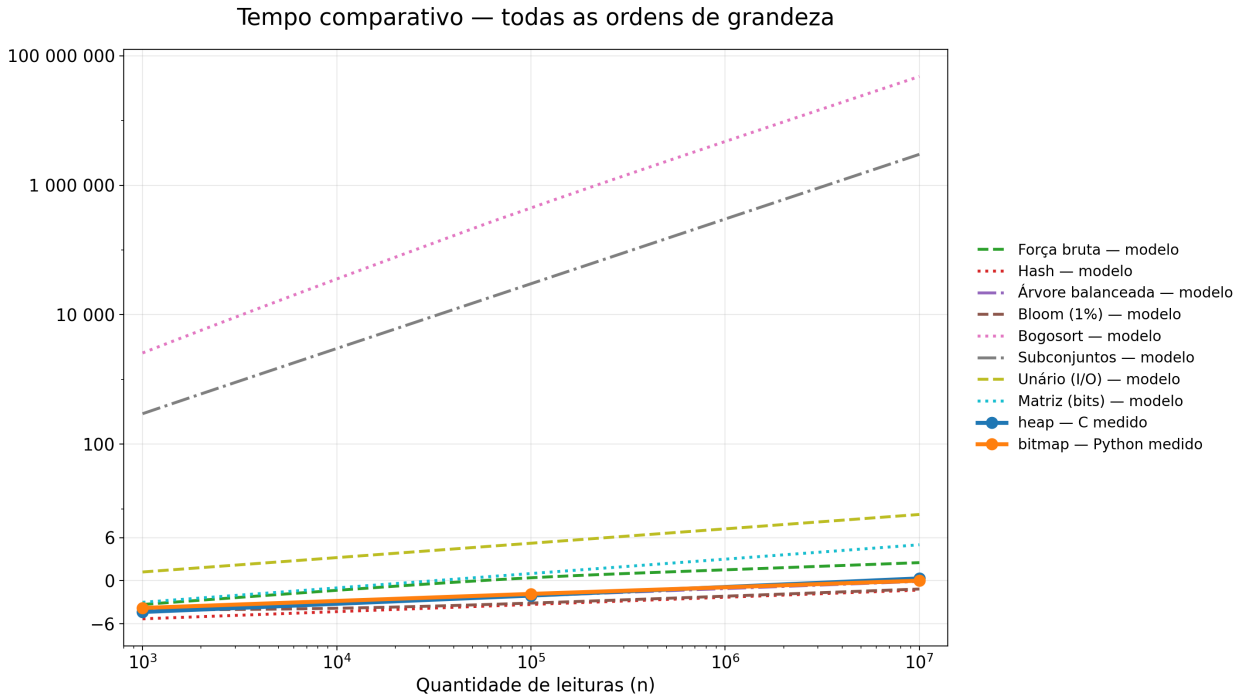
Recomendação: Heap Sort C para Embarca pelo limite local de memória comprovado; bitmap Python para Datacenter pelo domínio conhecido e custo linear, sujeito à referência. O prazo de dois segundos não é requisito Embarca: o tempo observado do Heap Sort é relatado integralmente, mesmo quando supera dois segundos. A comparação usa implementações em linguagens diferentes; a razão dos tempos não isola o efeito exclusivo dos algoritmos. Permanecem necessárias aprovação do par, confirmação da interpretação de linguagem diante das menções a Pascal e validação na máquina da disciplina.



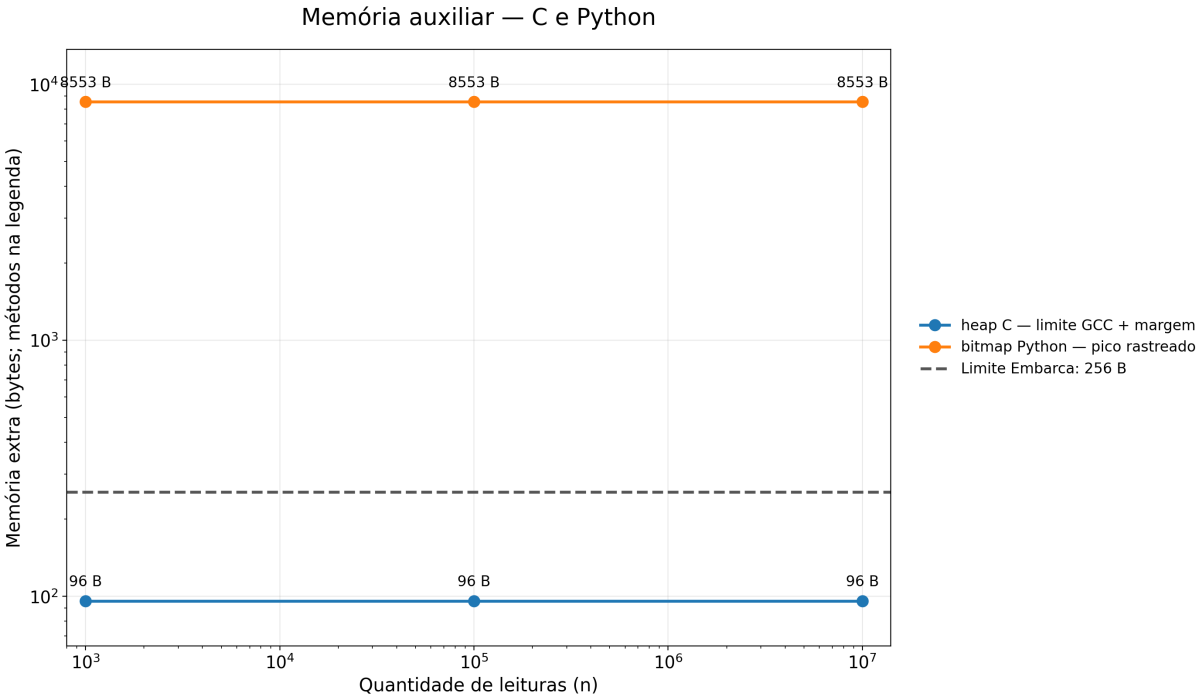
Pontos: medianas de 3 rodadas; barras: mínimo e máximo. Medições locais.



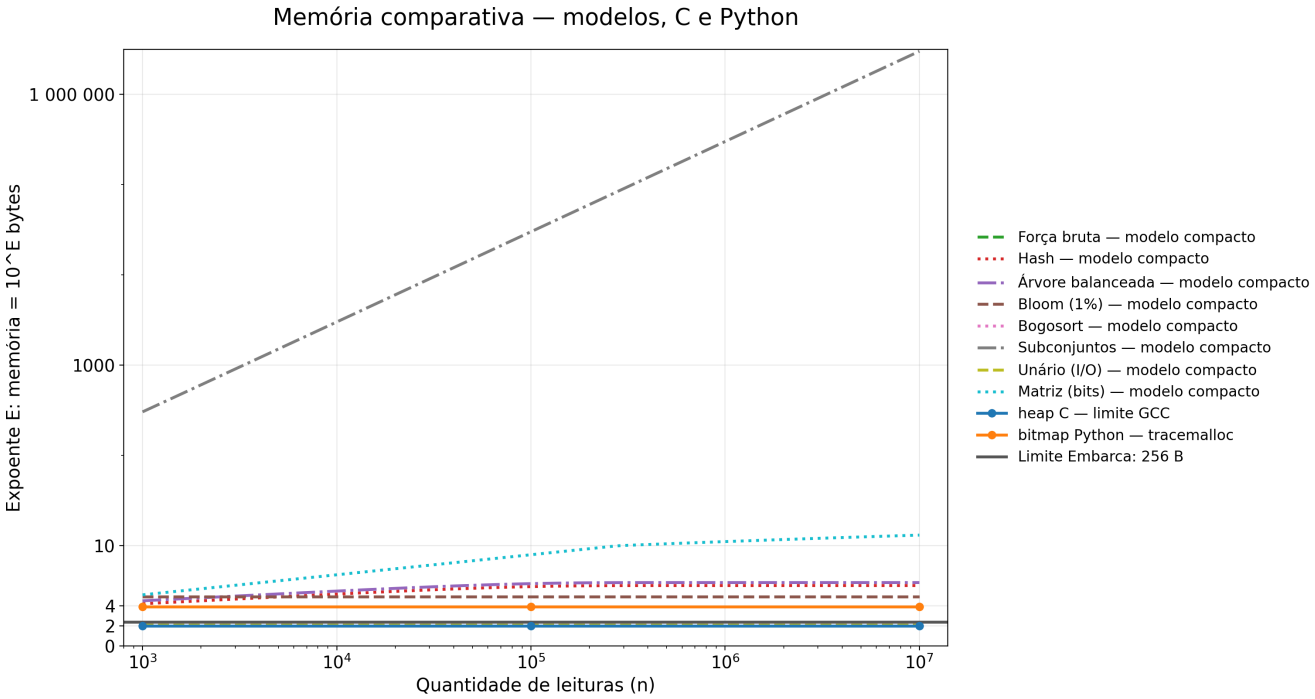
Modelos hipotéticos de baixo nível, não calibrados para C/Python. Curvas fora da faixa em tempo_completo.png.



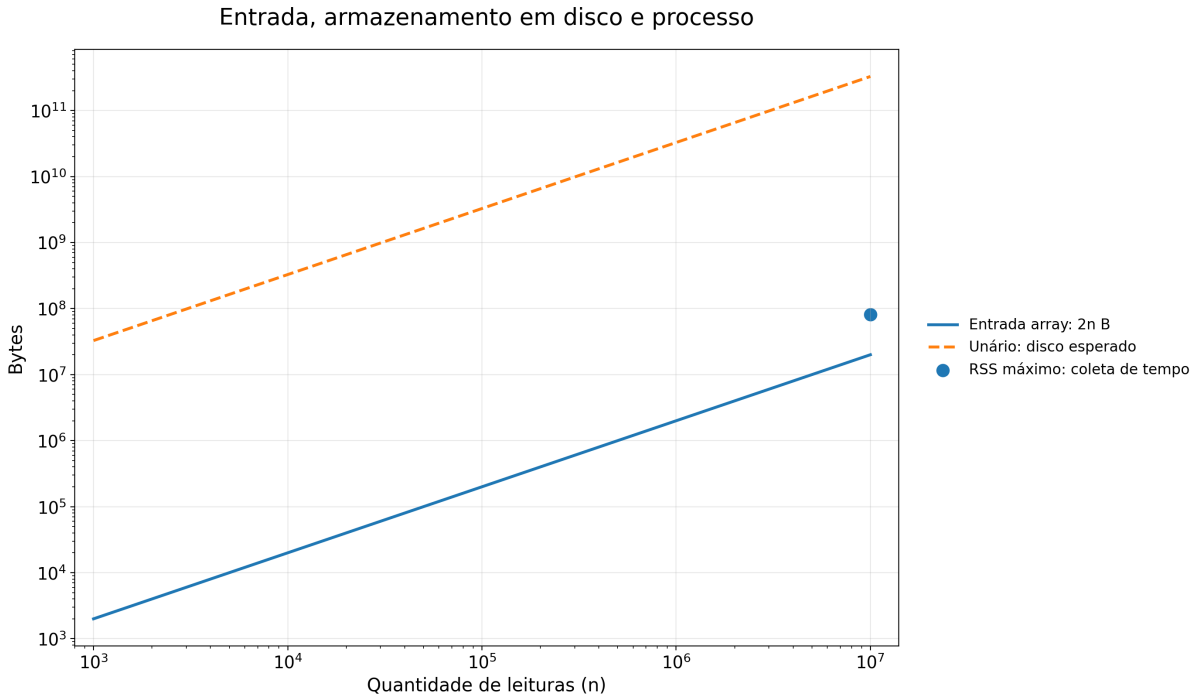
E é um expoente, não uma duração. Eixo comprimido para manter as oito curvas, inclusive fatoriais e exponenciais.



C: limite estático do algoritmo nativo. Python: pico rastreado. Valores não são RSS do processo.



C: limite GCC válido para todo n. Python: três picos rastreados. Alternativas tracejadas são modelos.



RSS: processo completo, incluindo infraestrutura. Não é memória extra de uma função.